



CHECKER

NAME : PENG SEYHA

S-CODE : s3

UNIT CODE : TCI-2510-CAMBODIA-SOC

INSTRUCTOR : RONA K SINGH

MENTOR : EM THARY

Table of Contents

1.Introduction	2
2.Project Overview	3
3.Tools and Technologies	3
4.System Architecture and Implementation	4
4.1 Environment Setup	4
4.2 Network Range Selection	5
4.3 Host Discovery and Target Selection	5
4.4 Safe Attack Simulation Engine	6
5.Attack Simulation Modules	7
5.1 Port Scanning Detection	7
5.2 DoS Attack Simulation	9
5.3 Brute Force Login Detection	10
5.4 Malware Detection Simulation	11
6.Dashboard and Log Management	13
6.1 SOC Executive Dashboard	13
6.2 Alert Generation and Severity Classification	13
6.3 Incident Logging and IOC Export	14
7.Testing and Results	15
7.1 Attack Simulation Testing	16
7.2 Dashboard and Log Verification	17
8.Security Considerations and Future Improvements	17
9.Conclusion	17
10.References	18

1. Introduction



CyberShield Simulator is a Bash-based project developed in Kali Linux to simulate Security Operations Center (SOC) activities in a safe and controlled environment. The purpose of this project is to demonstrate how SOC analysts detect, analyze, and respond to cybersecurity threats. The system performs network scanning, identifies active hosts, and simulates various attack scenarios. It generates SOC alerts that include incident IDs, severity levels, MITRE ATT&CK mapping, and risk scores. All activities are logged for further analysis and investigation.

This project is designed strictly for educational purposes. It does not perform real harmful attacks, but instead replicates realistic attack behaviors to improve understanding of SOC workflows, incident response, and security monitoring practices.



Figure 1. CyberShield SOC Simulator process workflow.

2. Project Overview

CyberShield Simulator provides a practical environment for learning SOC operations. It allows users to scan networks, select targets, simulate attacks, and analyze generated alerts. The system produces realistic SOC outputs, including:

- Incident identification
- Severity classification
- Risk scoring
- MITRE ATT&CK mapping
- Analyst investigation steps

3. Tools and Technologies

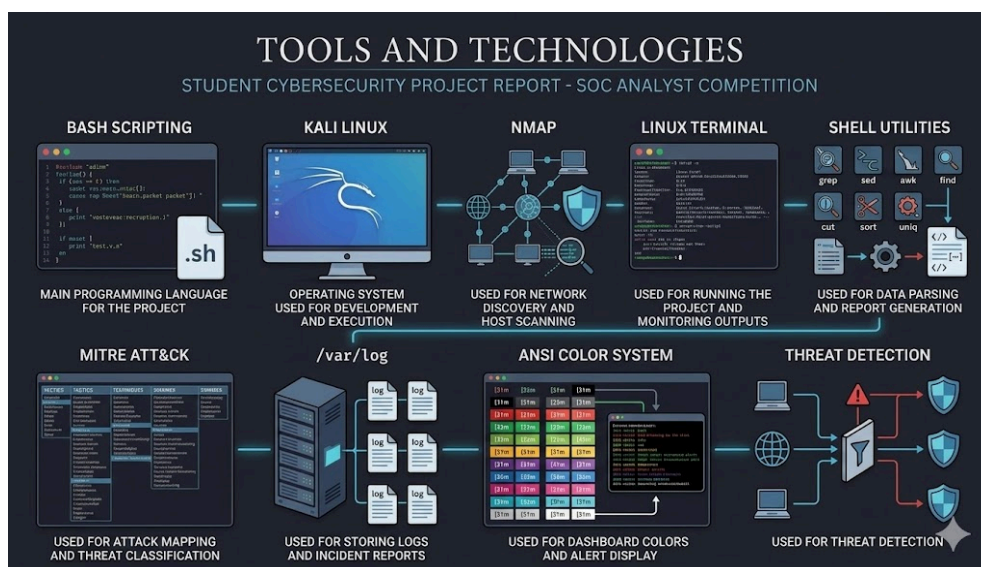


Figure 1. Tools and technologies used in the CyberShield Simulator project.

- Bash Scripting – main programming language for the project
- Kali Linux – operating system used for development and execution
- Nmap – used for network discovery and host scanning
- Linux Terminal – used for running the project and monitoring outputs
- /var/log – used for storing logs and incident reports
- MITRE ATT&CK – used for attack mapping and threat classification
- ANSI Color System – used for dashboard colors and alert display
- Shell Utilities – used for data parsing and report generation

4. System Architecture and Implementation

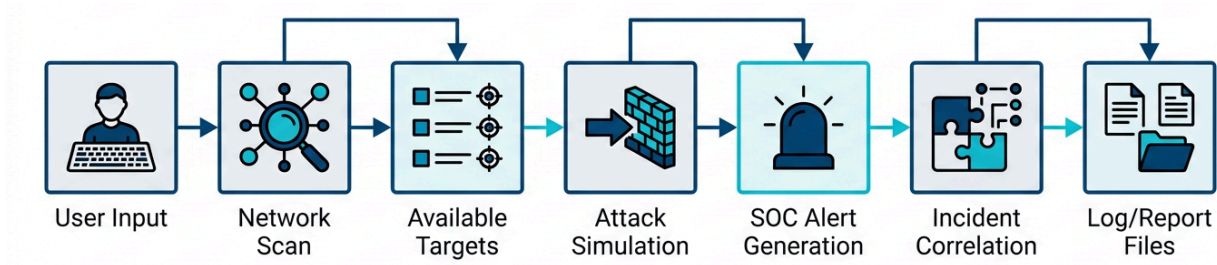


Figure 2. System architecture and workflow of the CyberShield SOC Simulator.

4.1 Environment Setup

Before starting the simulation, the system prepares the working environment required for the project. Since CyberShield Simulator is developed using Bash Script in Kali Linux, the terminal environment must be ready first.

The system automatically creates the required folders and log files inside `/var/log/cybershield/` to store incidents, reports, IOC exports, and session summaries. This is important because every simulation must be recorded for later investigation and final reporting.

```
(kali㉿kali)-[~]
$ sudo tree /var/log/cybershield/
/var/log/cybershield/
├── correlation.log
├── incidents.log
├── ioc_export.txt
├── reports.log
└── summary.log
```

Figure 3. Log directory structure of CyberShield Simulator showing incident logs, reports, and IOC export files.

```
setup_log_paths() {
    if [ -w /var/log ] 2>/dev/null || [ "${id -u}" -eq 0 ]; then
        LOG_DIR="/var/log/cybershield"
    else
        LOG_DIR="$HOME/.cybershield/logs"
    fi
    mkdir -p "$LOG_DIR" 2>/dev/null || { echo "Cannot create log directory: $LOG_DIR"; exit 1; }
    chmod 700 "$LOG_DIR" 2>/dev/null || true
    LOG_FILE="$LOG_DIR/incidents.log"
    REPORT_FILE="$LOG_DIR/reports.log"
    SUMMARY_FILE="$LOG_DIR/summary.log"
    CORRELATION_FILE="$LOG_DIR/correlation.log"
    IOC_FILE="$LOG_DIR/ioc_export.txt"
    local f
    for f in "$LOG_FILE" "$REPORT_FILE" "$SUMMARY_FILE" "$CORRELATION_FILE" "$IOC_FILE"; do
        touch "$f" 2>/dev/null || { echo "Cannot create: $f"; exit 1; }
        chmod 600 "$f" 2>/dev/null || true
    done
}
```

Code Snippet1. Environment setup function for preparing CyberShield Simulator log files.

4.2 Network Range Selection

After the environment setup, the system allows the user to select the target network range for scanning. Three options are provided: auto-detected LAN range, custom CIDR range, and single host mode.

This step helps define the scanning scope before host discovery begins. It makes the simulator more flexible and closer to real SOC investigation workflows.

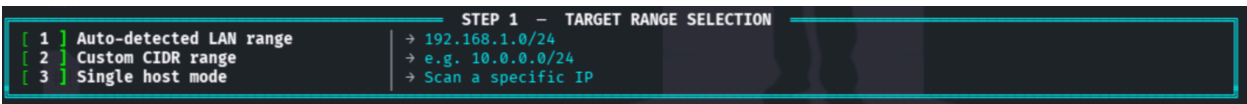


Figure 4. Network range selection menu of CyberShield Simulator.

```
select_network_range() {  
    local range choice custom single  
    range=$(get_current_network_range)  
    CURRENT_NETWORK_RANGE="$range"  
    show_network_menu "$range"  
    read_menu_choice "Select range (1-3):" choice  
    case "$choice" in  
        1) if [ -n "$range" ]; then  
            SELECTED_SCAN_RANGE="$range"  
            status_ok "Range: ${C_WHITE}${range}${NC}"  
        else  
            status_error "Cannot auto-detect range."  
            return 1  
        fi ;;  
        2) safe_read "Enter CIDR (e.g. 192.168.1.0/24):" custom  
        if validate_range "$custom"; then  
            SELECTED_SCAN_RANGE="$custom"  
            status_ok "Custom range: ${C_WHITE}${custom}${NC}"  
        else  
            status_error "Invalid CIDR."  
            return 1  
        fi ;;  
        3) safe_read "Enter host IP:" single  
        if validate_ip "$single"; then  
            SELECTED_SCAN_RANGE="${single}/32"  
            status_ok "Single host: ${C_WHITE}${single}${NC}"  
        else  
            status_error "Invalid IP."  
            return 1  
        fi ;;  
        *) status_error "Invalid choice."; return 1 ;;  
    esac  
}
```

Code Snippet 2. Network range selection function used for selecting scan scope.

4.3 Host Discovery and Target Selection

After selecting the network range, the system uses Nmap to discover active hosts on the network. This helps identify available devices that can be used as targets for simulation and investigation. The user can then select a target manually, use the current machine, enter a custom IP address, or allow random target selection. This makes the simulator more practical and similar to real SOC operations.

STEP 2 - DISCOVERED HOSTS				
ID	HOST ADDRESS	IP	STATUS	CLASSIFICATION
[01]	192.168.1.1	192	● ONLINE	Simulation Target
[02]	192.168.1.10	192	● ONLINE	Simulation Target
[03]	192.168.1.14	192	● ONLINE	Simulation Target
[04]	192.168.1.17	192	● ONLINE	Simulation Target
[05]	192.168.1.2	192	● ONLINE	Simulation Target
[06]	192.168.1.20	192	● ONLINE	Simulation Target
[07]	192.168.1.22	192	● ONLINE	Simulation Target
[08]	192.168.1.27	192	● ONLINE	Simulation Target
[09]	192.168.1.28	192	● ONLINE	Simulation Target
[10]	192.168.1.29	192	● ONLINE	Simulation Target
[11]	192.168.1.30	192	● ONLINE	Simulation Target
[12]	192.168.1.32	192	● ONLINE	Simulation Target
[13]	192.168.1.37	192	● ONLINE	Simulation Target
[14]	192.168.1.4	192	● ONLINE	Simulation Target
[15]	192.168.1.57	192	● ONLINE	Simulation Target

Figure 5. Shows the host discovery result with active IP addresses detected by the system.

CURRENT SESSION CONTEXT	
NETWORK RANGE	192.168.1.0/24 18 hosts scanned
LAST ACTIVITY	None yet
LOG DIRECTORY	/var/log/cybershield
TARGET HOST	Not selected
SESSION RISK	0 / 100 Safe
LOG FILES	5 files

STEP 3 - SELECT SIMULATION TARGET	
[1] Current machine (self)	→ 192.168.1.14
[2] Choose from discovered list	→ Select by ID
[3] Enter custom IP	→ Any valid address
[4] Auto-select random target	→ Randomly picked

Figure 6. Shows the target selection menu used for choosing the simulation target.

4.4 Safe Attack Simulation Engine

After selecting a target, the system executes safe attack simulations such as Port Scanning, DoS behavior, Brute Force attempts, and Malware activity.

These simulations do not perform real attacks. Instead, they generate realistic behavioral patterns that trigger SOC alerts. Each alert includes:

- Unique incident ID
- Severity level
- MITRE ATT&CK mapping
- Risk score
- Recommended analyst response

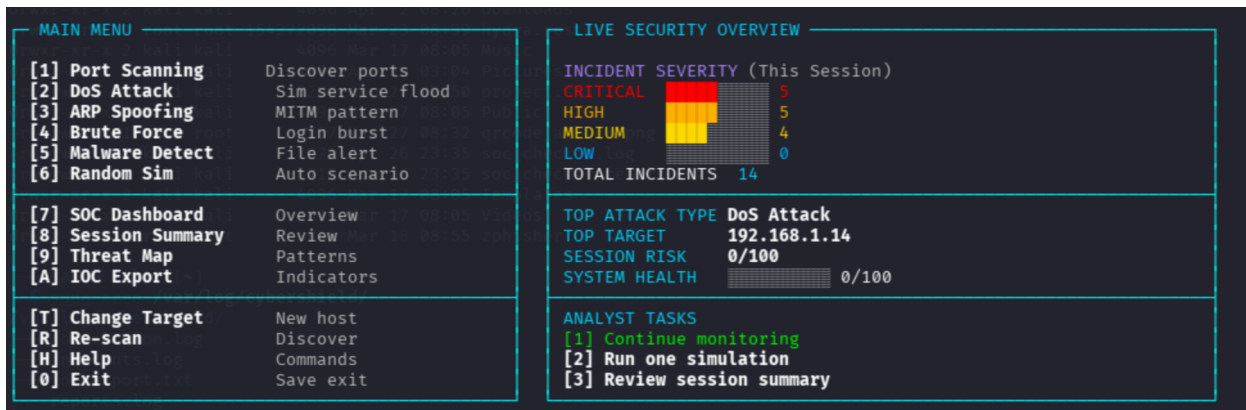


Figure 7. Shows the attack selection menu used for choosing the simulation type.

```

> execute_attack() {
    local choice="$1" target="$2" actual_choice="$1"
    case "$choice" in
        1) sim_port_scan "$target" ;;
        2) sim_dos "$target" ;;
        3) sim_arp_spoof "$target" ;;
        4) sim_brute_force "$target" ;;
        5) sim_malware "$target" ;;
        6)
            local r
            r=$((1 + RANDOM % 5))
            actual_choice="$r"
            status_info "Auto-selected module: ${C_WHITE}${(get_attack_name "$r")}${NC}"
            case "$r" in
                1) sim_port_scan "$target" ;;
                2) sim_dos "$target" ;;
                3) sim_arp_spoof "$target" ;;
                4) sim_brute_force "$target" ;;
                5) sim_malware "$target" ;;
            esac ;;
        *) status_error "Invalid module."; return 1 ;;
    esac
    SESSION_LAST_ATTACK=$(get_attack_name "$actual_choice")
}

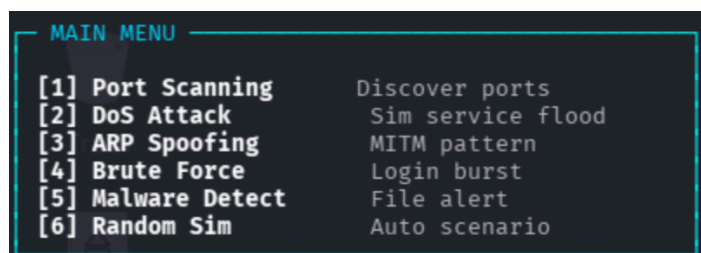
```

Code Snippet 3. Attack execution function used for running safe attack simulations .

5. Attack Simulation Modules

The system simulates attacks such as Port Scanning, DoS attacks, Brute Force Login attempts, and Malware Detection. Instead of performing real harmful attacks, the simulator safely generates alerts, severity levels, MITRE ATT&CK mapping, risk scores, and analyst response steps.

Figure 8. Attack simulation menu of CyberShield Simulator .



5.1. Port Scanning Detection

Port scanning is commonly used by attackers to identify open services and potential vulnerabilities.

In this module, CyberShield simulates port scanning behavior and generates a SOC alert. The alert includes incident details, severity level, and risk score. The simulation was validated using Wireshark by observing TCP SYN packet activity in a controlled lab environment.

SIMULATION RESULTS - PORT SCAN

Ports Probed: 25
Exposed Services: 22/tcp ssh 80/tcp http 443/tcp https
Evidence: 25 probes exceeded detection threshold.

Threshold: 15

SECURITY ALERT

INCIDENT ALERT REPORT

Incident ID: SOC-20260429-093803-14003
Target Host: 192.168.1.18
Severity: Medium (was: Medium)
Repeat Count: 2x
MITRE ATT&CK: T1046 - Network Service Discovery

Attack Type: Port Scanning
Detection: Network IDS / Firewall Logs
Risk Score: 51 / 100
Status: OPEN - Pending Analyst Review

Risk Level: 51/100

ANALYST ASSESSMENT

Why Alert Fired: 25 port probes exceeded threshold of 15.
Root Cause: Pre-exploitation reconnaissance activity suspected.
Recommendation: Review firewall rules and confirm scan authorization.

INCIDENT TIMELINE

09:38:04: Anomalous activity detected on 192.168.1.18
09:38:04: Detection threshold crossed - alert generated
09:38:04: Severity: Medium Risk Score: 51/100

IOC SNAPSHOT

Target IP: 192.168.1.18
Source: Network IDS / Firewall Logs

Attack Type: Port Scanning
Risk Score: 51/100

TIER 1 TRIAGE WORKFLOW

01. Verify if source IP matches any authorized security scanner.
02. Review firewall and IDS logs for source IP.
03. Identify and close unnecessary listening ports.
04. Escalate if source is unknown or scan repeats.

ANALYST DECISION

☒ Escalate / Review ☐ False Positive ☐ Monitor Only ☐ Contained

Figure 9. Shows the Port Scanning alert generated by CyberShield Simulator.

ip.src == 192.168.1.19 && tcp.flags.syn == 1

No.	Time	Source	Destination	Protocol	Length	Info
94	28.062246219	192.168.1.19	192.168.1.18	TCP	60	43324 → 3396 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
96	28.062259520	192.168.1.19	192.168.1.18	TCP	60	43324 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
98	28.062271813	192.168.1.19	192.168.1.18	TCP	60	43324 → 8888 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
100	28.062282580	192.168.1.19	192.168.1.18	TCP	60	43324 → 993 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
102	28.062296625	192.168.1.19	192.168.1.18	TCP	60	43324 → 139 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
107	28.063929869	192.168.1.19	192.168.1.18	TCP	60	43324 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
109	28.063967408	192.168.1.19	192.168.1.18	TCP	60	43324 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
111	28.063988827	192.168.1.19	192.168.1.18	TCP	60	43324 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
113	28.063997421	192.168.1.19	192.168.1.18	TCP	60	43324 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
115	28.064016717	192.168.1.19	192.168.1.18	TCP	60	43324 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
117	28.064025112	192.168.1.19	192.168.1.18	TCP	60	43324 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
119	28.064034251	192.168.1.19	192.168.1.18	TCP	60	43324 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
121	28.064043825	192.168.1.19	192.168.1.18	TCP	60	43324 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
123	28.064051944	192.168.1.19	192.168.1.18	TCP	60	43324 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
125	28.064061265	192.168.1.19	192.168.1.18	TCP	60	43324 → 110 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
127	28.064069251	192.168.1.19	192.168.1.18	TCP	60	43324 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
129	28.064078650	192.168.1.19	192.168.1.18	TCP	60	43324 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
131	28.064086722	192.168.1.19	192.168.1.18	TCP	60	43324 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
133	28.064097647	192.168.1.19	192.168.1.18	TCP	60	43324 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Frame 115: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface
Ethernet II, Src: PCShield03:08:05 (08:00:27:63:b0:05), Dst: Intel_44:00:00:00:00:00
Internet Protocol Version 4, Src: 192.168.1.19, Dst: 192.168.1.18
Transmission Control Protocol, Src Port: 43324, Dst Port: 80, Seq: 0, Len: 0

Figure 10. Shows Wireshark packet capture during Port Scanning validation between Kali Linux and the Ubuntu target system.

```

sim_port_scan() {
    local target="$1" ports sev
    ports=$((20 + RANDOM % 80))
    sev="Medium"; [ "$ports" -gt 60 ] && sev="High"
    status_warn "Port Scan on ${C_WHITE}${target}${NC}"
    ui_loading_bar "Enumerating services on ${target}" 22
    ui_box_top "$SUI_WIDTH" "SIMULATION RESULTS - PORT SCAN"
    ui_metric_row "$SUI_WIDTH" "Ports Probed" "${C_WHITE}${ports}${NC}" "Threshold" "${C_WHITE}15${NC}"
    ui_box_line "$SUI_WIDTH" "${C_SUCCESS}Exposed Services${NC}" "${C_PRIMARY}22/tcp ssh 80/tcp http 443/tcp https${NC}"
    ui_box_line "$SUI_WIDTH" "${C_GRAY}Evidence${NC}" "${C_WHITE}${ports} probes exceeded detection threshold.${NC}"
    ui_box_bottom "$SUI_WIDTH"
    create_soc_alert "Port Scanning" "$target" "$sev" \
        "Network IDS / Firewall Logs" "Open" "T1046 - Network Service Discovery" \
        "${ports} port probes exceeded threshold of 15." \
        "Pre-exploitation reconnaissance activity suspected." \
        "Review firewall rules and confirm scan authorization." \
        "Verify if source IP matches any authorized security scanner." \
        "Review firewall and IDS logs for source IP." \
        "Identify and close unnecessary listening ports." \
        "Escalate if source is unknown or scan repeats."
}

```

Code Snippet 5. Shows the function used for simulating Port Scanning detection and generating the SOC alert.

5.2. DoS Attack Simulation

Denial-of-Service (DoS) attacks attempt to overwhelm a system with excessive traffic, making it unavailable to legitimate users.

This module simulates abnormal traffic patterns and generates corresponding SOC alerts. The behavior was validated using repeated ICMP traffic observed through Wireshark in a controlled environment.

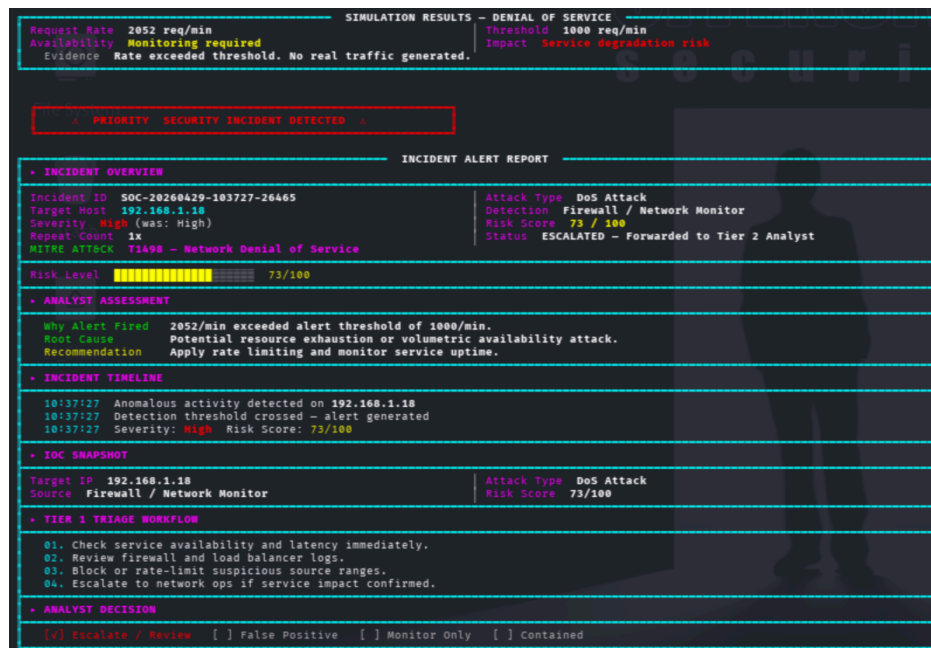


Figure 11. Shows the DoS Attack alert generated by CyberShield Simulator.

No.	Time	Source	Destination	Protocol	Length	Info
1094	3685.8577917	18.198.103.184	192.168.1.18	TCP	60	443 → 34882 [ACK] Seq=6086 Ack=8928 Win=64128 Len=0 TSval=3414357048 TS
1094	3686.4724794	18.198.103.184	192.168.1.18	TLSv1.2	109	Application Data
1094	3686.4730521	192.168.1.18	18.198.103.184	TLSv1.2	113	Application Data
1094	3686.6630615	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) request id=0x0008, seq=14/3584, ttl=64 (reply in 109461)
1094	3686.6631334	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) reply id=0x0008, seq=14/3584, ttl=64 (request in 109460)
1094	3686.7794349	18.198.103.184	192.168.1.18	TCP	66	443 → 34872 [ACK] Seq=6086 Ack=8994 Win=64128 Len=0 TSval=3414357968 TS
1094	3687.1888181	91.108.56.104	192.168.1.18	TCP	171	443 → 48172 [PSH, ACK] Seq=272072 Ack=88900 Win=21225 Len=105 TSval=285
1094	3687.2291756	192.168.1.18	91.108.56.104	TCP	66	48172 → 443 [ACK] Seq=88900 Ack=272177 Win=2092 Len=0 TSval=52551690 TS
1094	3687.4963484	13.107.5.93	192.168.1.18	TCP	54	443 → 57842 [RST, ACK] Seq=14682 Ack=4843 Win=0 Len=0
1094	3687.6632086	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) request id=0x0008, seq=15/3840, ttl=64 (reply in 109468)
1094	3687.6633041	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) reply id=0x0008, seq=15/3840, ttl=64 (request in 109467)
1094	3688.1279954	192.168.1.18	140.82.113.26	TCP	66	[TCP Keep-Alive] 43458 → 443 [ACK] Seq=7685 Ack=5445 Win=60672 Len=0 TS
1094	3688.4176868	140.82.113.26	192.168.1.18	TCP	66	[TCP Keep-Alive ACK] 443 → 43458 [ACK] Seq=5445 Ack=7686 Win=129024 Len
1094	3688.6795115	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) request id=0x0008, seq=16/4096, ttl=64 (reply in 109473)
1094	3688.6796011	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) reply id=0x0008, seq=16/4096, ttl=64 (request in 109472)
1094	3689.6815695	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) request id=0x0008, seq=17/4352, ttl=64 (reply in 109476)
1094	3689.6816382	192.168.1.18	192.168.1.18	ICMP	98	Echo (ping) reply id=0x0008, seq=17/4352, ttl=64 (request in 109475)
1094	3690.5172264	192.168.1.18	172.253.118.138	QUIC	71	Protected Payload (KPo), DCID=fac1c99a1c32e8a3
1094	3690.5956701	172.253.118.138	192.168.1.18	QUIC	68	Protected Payload (KPo)

Figure 12. Wireshark packet capture showing repeated ICMP traffic between Kali Linux and the Ubuntu target system during controlled DoS Attack simulation.

```

sim dos() {
    local target="$1" rate sev
    rate=$((800 + RANDOM % 2500))
    sev="High"; [ "$rate" -gt 2500 ] && sev="Critical"
    status warn "DoS simulation on ${C_WHITE}${target}${NC}"
    ui loading bar "Analyzing traffic flood pattern on ${target}" 22
    ui box top "$UI_WIDTH" "SIMULATION RESULTS - DENIAL OF SERVICE"
    ui metric row "$UI_WIDTH" "Request Rate" "${C_WHITE}${rate} req/min${NC}" "Threshold" "${C_WHITE}1000 req/min${NC}"
    ui metric row "$UI_WIDTH" "Availability" "${C_WARN}Monitoring required${NC}" "Impact" "${C_DANGER}Service degradation r
    ui box line "$UI_WIDTH" "${C_GRAY}Evidence${NC} ${C_WHITE}Rate exceeded threshold. No real traffic generated.${NC}"
    ui box bottom "$UI_WIDTH"
    create_soc alert "DoS Attack" "$target" "$sev" \
        "Firewall / Network Monitor" "Escalated - Tier 2" "T1498 - Network Denial of Service" \
        "${rate}/min exceeded alert threshold of 1000/min." \
        "Potential resource exhaustion or volumetric availability attack." \
        "Apply rate limiting and monitor service uptime." \
        "Check service availability and latency immediately." \
        "Review firewall and load balancer logs." \
        "Block or rate-limit suspicious source ranges." \
        "Escalate to network ops if service impact confirmed."
}

```

Code Snippet 5. Shows the function used for simulating DoS detection and generating the SOC alert.

5.3. Brute Force Login Detection

Brute force attacks involve repeated login attempts to gain unauthorized access. This module simulates multiple failed login attempts and triggers a SOC alert when suspicious activity is detected. The system generates incident details, severity classification, and investigation guidance.

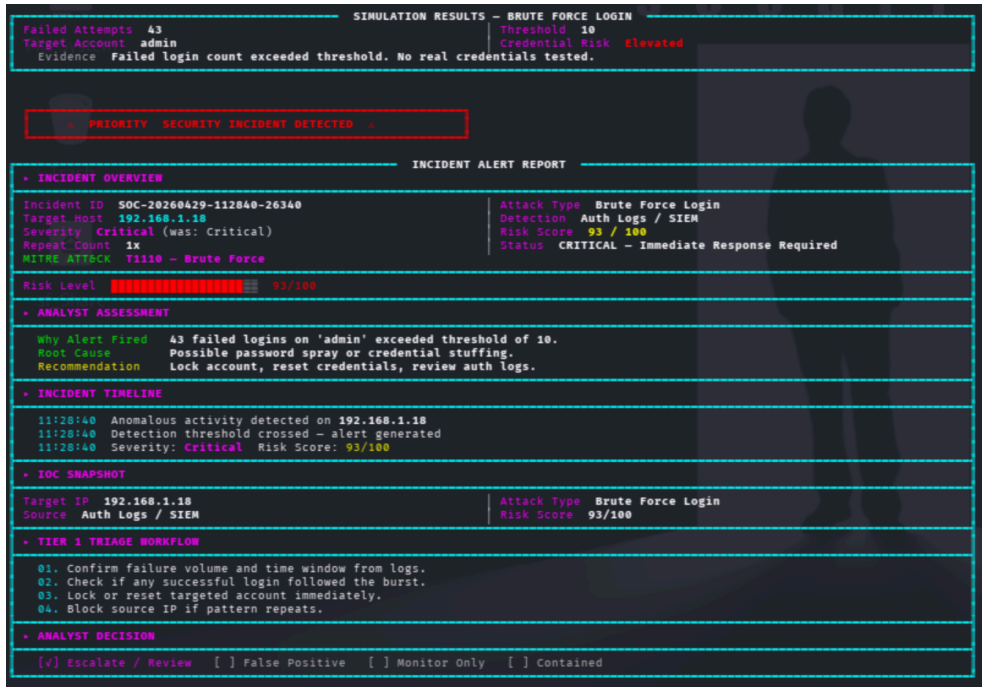


Figure 13. Shows the Brute Force Login alert generated by CyberShield Simulator.

```
sim brute force() {
  local target="$1" fails sev
  fails=$((10 + RANDOM % 40))
  sev="High"; [ "$fails" -gt 30 ] && sev="Critical"
  status warn "Brute Force Simulation on ${C_WHITE}${target}${NC}"
  ui loading_bar "Analyzing failed authentication on ${target}" 22
  ui_box_top "$SUI_WIDTH" "SIMULATION RESULTS - BRUTE FORCE LOGIN"
  ui_metric_row "$SUI_WIDTH" "Failed Attempts" "${C_WHITE}${fails}${NC}" "Threshold" "${C_WHITE}10${NC}"
  ui_metric_row "$SUI_WIDTH" "Target Account" "${C_WHITE}admin${NC}" "Credential Risk" "${C_DANGER}Elevated${NC}"
  ui_box_line "$SUI_WIDTH" "${C_GRAY}Evidence${NC} ${C_WHITE}Failed login count exceeded threshold. No real credentials tested."
  ui_box_bottom "$SUI_WIDTH"
  create_soc alert "Brute Force Login" "$target" "$sev" \
    "Auth Logs / SIEM" "Escalated - Tier 2" "T1110 - Brute Force" \
    "${fails} failed logins on 'admin' exceeded threshold of 10." \
    "Possible password spray or credential stuffing." \
    "Lock account, reset credentials, review auth logs." \
    "Confirm failure volume and time window from logs." \
    "Check if any successful login followed the burst." \
    "Lock or reset targeted account immediately." \
    "Block source IP if pattern repeats."
}
```

Code Snippet 6. Shows the function used for simulating Brute Force detection and generating the SOC

5.4. Malware Detection Simulation

Malware can cause serious damage to systems and data. This module simulates malicious activity and generates a high-severity SOC alert. It demonstrates how analysts identify and respond to critical threats quickly.

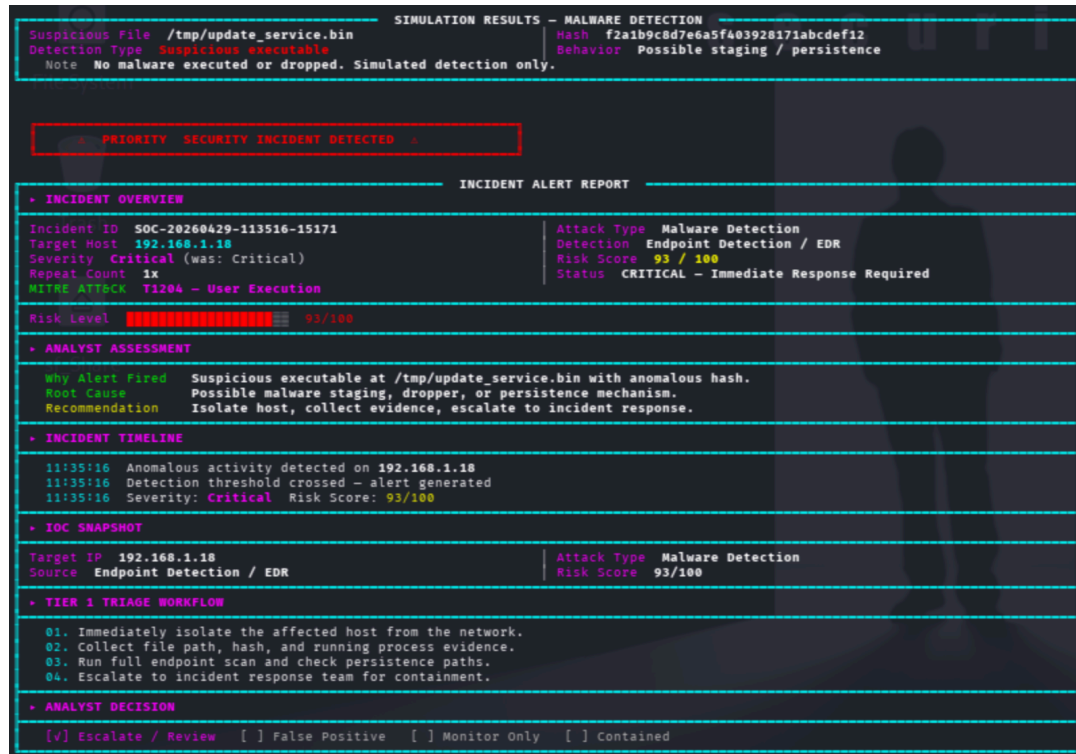


Figure 14. Shows the Malware Detection alert generated by CyberShield Simulator.

```
sim malware() {
  local target="$1" fake_hash="f2a1b9c8d7e6a5f403928171abcdef12"
  status warn "Malware Detection Simulation on ${C_WHITE}${target}${NC}"
  ui_loading_bar "Scanning file system for suspicious executables on ${target}" 22
  ui_box_top "$SUI_WIDTH" "SIMULATION RESULTS - MALWARE DETECTION"
  ui_metric_row "$SUI_WIDTH" "Suspicious File" "${C_WHITE}/tmp/update_service.bin${NC}" "Hash" "${C_WHITE}${fake_hash}${NC}"
  ui_metric_row "$SUI_WIDTH" "Detection Type" "${C_DANGER}Suspicious executable${NC}" "Behavior" "${C_WHITE}Possible staging${NC}"
  ui_box_line "$SUI_WIDTH" "${C_GRAY}Notes${NC} ${C_WHITE}No malware executed or dropped. Simulated detection only.${NC}"
  ui_box_bottom "$SUI_WIDTH"
  create_soc_alert "Malware Detection" "$target" "Critical" \
    "Endpoint Detection / EDR" "CRITICAL - Immediate Response" "T1204 - User Execution" \
    "Suspicious executable at /tmp/update_service.bin with anomalous hash." \
    "Possible malware staging, dropper, or persistence mechanism." \
    "Isolate host, collect evidence, escalate to incident response." \
    "Immediately isolate the affected host from the network." \
    "Collect file path, hash, and running process evidence." \
    "Run full endpoint scan and check persistence paths." \
    "Escalate to incident response team for containment."
}
```

Code Snippet 8. The function used for simulating Malware detection and generating the SOC alert.

6. Dashboard and Log Management

6.1 SOC Executive Dashboard

The dashboard shows important information such as detected hosts, attack type, severity level, MITRE ATT&CK mapping, and investigation status. This makes incident response faster and improves visibility during security monitoring.

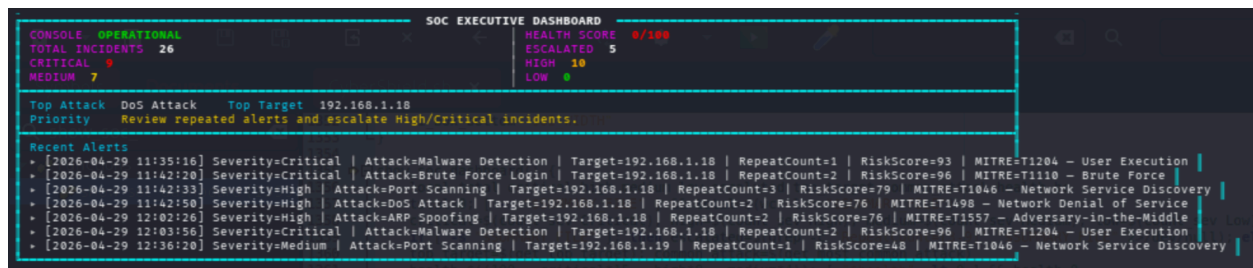


Figure 15. SOC Executive Dashboard showing alerts, risk score, and incident status.

```
show soc dashboard() {
    local total critical high medium low escalated top_target common attack health hc
    total=0; [ -s "$SUMMARY_FILE" ] && total=$(wc -l < "$SUMMARY_FILE")
    critical=$(count sev Critical); high=$(count sev High); medium=$(count sev Medium); low=$(count sev Low)
    if [ -s "$SUMMARY_FILE" ]; then escalated=$(grep -c "RepeatCount=[3-9]" "$SUMMARY_FILE" 2>/dev/null); else escalated=0; fi
    top_target=$(get_top target); common_attack=$(get_most common attack)
    health=$((100 - critical*15 - high*8 - medium*4)); [ "$health" -lt 0 ] && health=0
    hc="$C_GREEN"; [ "$health" -lt 70 ] && hc="$C_YELLOW"; [ "$health" -lt 40 ] && hc="$C_RED"
    ui_topbar
    ui_box_top "$SUI_WIDTH" "SOC EXECUTIVE DASHBOARD"
    ui_metric_row "$SUI_WIDTH" "CONSOLE" "${C_GREEN}OPERATIONAL${NC}" "HEALTH SCORE" "${hc}${health}/100${NC}"
    ui_metric_row "$SUI_WIDTH" "TOTAL INCIDENTS" "$total" "ESCALATED" "$escalated"
    ui_metric_row "$SUI_WIDTH" "CRITICAL" "${C_RED}${critical}${NC}" "HIGH" "${C_ORANGE}${high}${NC}"
    ui_metric_row "$SUI_WIDTH" "MEDIUM" "${C_YELLOW}${medium}${NC}" "LOW" "${C_GREEN}${low}${NC}"
    ui_box_separator "$SUI_WIDTH"
    ui_box_line "$SUI_WIDTH" "${C_NEON}Top Attack${NC} ${common_attack} ${C_NEON}Top Targets${NC} ${top_target}"
    ui_box_line "$SUI_WIDTH" "${C_NEON}Priority${NC} ${C_YELLOW}Review repeated alerts and escalate High/Critical incidents.${NC}"
    ui_box_separator "$SUI_WIDTH"
    ui_box_line "$SUI_WIDTH" "${C_NEON}Recent Alerts${NC}"
    if [ -s "$SUMMARY_FILE" ]; then
        tail -n 7 "$SUMMARY_FILE" | while IFS= read -r line; do ui_box_line "$SUI_WIDTH" "${C_GRAY}${line}${NC}"; done
    else
        ui_box_line "$SUI_WIDTH" "${C_GRAY}No incidents yet. Run a simulation first.${NC}"
    fi
    ui_box_bottom "$SUI_WIDTH"
}
```

Code Snippet 9: Function used to display the SOC dashboard and incident summary.

6.2 Alert Generation and Severity Classification

Severity classification helps SOC analysts prioritize incidents effectively.

- Low-level alerts require monitoring
- Medium-level alerts require investigation
- High and Critical alerts require immediate response

This classification improves decision-making and ensures efficient incident handling.

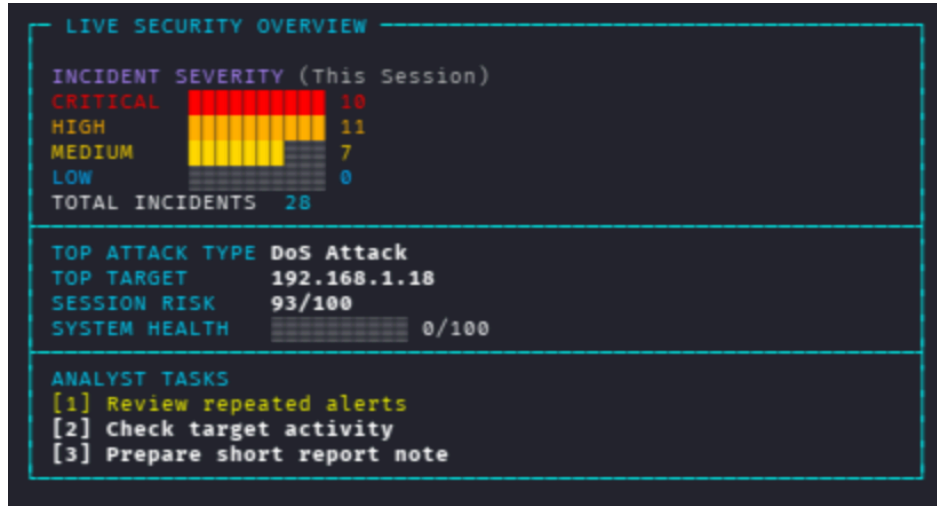


Figure 15. Alert severity classification showing Low, Medium, High, and Critical levels.

```

}create_soc_alert() {
    local attack="$1" target="$2" sev="$3" src="$4" status="$5" mitre="$6"
    local why="$7" cause="$8" action="$9" t1="${10}" t2="${11}" t3="${12}" t4="${13}"
    local id rc rcount orig_sev risk
    id=$(generate_incident_id)
    rc=$(get_repeat_count "$attack" "$target")
    rcount=$((rc+1))
    orig_sev="$sev"
    sev=$(escalate_severity "$sev" "$rcount")
    status=$(get_alert_status "$sev" "$rcount")
    risk=$(calculate_risk_score "$sev" "$rcount")
    if [ "$rcount" -ge 3 ]; then
        why="$why [Repeated: ${rcount}x]"
        cause="$cause Correlation engine flagged repeated activity."
        action="$action PRIORITY: escalate immediately."
    fi
    print_soc_alert "$id" "$attack" "$target" "$sev" "$src" "$status" "$mitre" \
        "$why" "$cause" "$action" "$t1" "$t2" "$t3" "$t4" "$rcount" "$risk" "$orig_sev"
    save_soc_record "$id" "$attack" "$target" "$sev" "$src" "$status" "$mitre" \
        "$why" "$cause" "$action" "$t1" "$t2" "$t3" "$t4" "$rcount" "$risk" "$orig_sev"
    record_correlation_event "$attack" "$target" "$sev" "$id"
    SESSION_LAST_SEVERITY="$sev"
    SESSION_LAST_RISK="$risk"
    echo ""
    status_ok "Incident saved → ${C_WHITE}${LOG_FILE}${NC}"
    status_ok "IOC exported → ${C_WHITE}${IOC_FILE}${NC}"
}
  
```

Code Snippet 10: Function used to generate alerts and assign severity levels.

6.3 Incident Logging and IOC Export Testing and Results

All simulation results are automatically saved inside `/var/log/cybershield/` to support reporting and forensic investigation. The system stores incidents, reports, summaries, correlation logs, and IOC export files for analyst review.

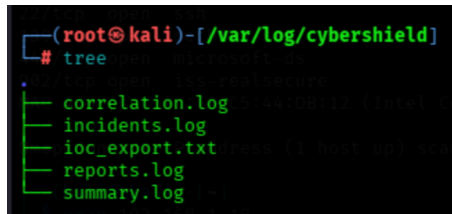


Figure 17. CyberShield log files stored in /var/log/cybershield/.

IOC EXPORT – INDICATORS OF COMPROMISE				
TIMESTAMP	TARGET	ATTACK	RISK	ID
2026-04-29 11:28:40	192.168.1.18	Brute Force Login	93	SOC-20260429-112840-26340
2026-04-29 11:35:16	192.168.1.18	Malware Detection	93	SOC-20260429-113516-15171
2026-04-29 11:42:20	192.168.1.18	Brute Force Login	96	SOC-20260429-114220-12539
2026-04-29 11:42:33	192.168.1.18	Port Scanning	79	SOC-20260429-114233-16633
2026-04-29 11:42:50	192.168.1.18	DoS Attack	76	SOC-20260429-114249-32606
2026-04-29 12:02:26	192.168.1.18	ARP Spoofing	76	SOC-20260429-120226-20573
2026-04-29 12:03:56	192.168.1.18	Malware Detection	96	SOC-20260429-120356-20414
2026-04-29 12:36:20	192.168.1.19	Port Scanning	48	SOC-20260429-123620-23001
2026-04-29 12:37:24	192.168.1.19	Brute Force Login	73	SOC-20260429-123724-12088
2026-04-29 12:57:01	192.168.1.19	Malware Detection	93	SOC-20260429-125701-24850

Full export: /var/log/cybershield/ioc_export.txt

Figure 18: IOC export showing attack records and risk scores.

```
show_ioc_export() {
  echo ""
  ui_box_top "$SUI_WIDTH" "IOC EXPORT – INDICATORS OF COMPROMISE"
  if [ ! -s "$IOC_FILE" ]; then
    ui_box_line "$SUI_WIDTH" " ${C_GRAY}No IOCs recorded yet.${NC}"
    ui_box_bottom "$SUI_WIDTH"
    return
  fi
  ui_box_line "$SUI_WIDTH" " ${C_ACCENT}TIMESTAMP          TARGET          ATTACK          RISK  ID${NC}"
  ui_box_separator "$SUI_WIDTH"
  tail -n 10 "$IOC_FILE" | while IFS= read -r line; do
    local ts tgt atk risk iid
    ts=$(echo "$line" | sed -n 's/^\[([^\]]*\)]\.[^\1/p')
    tgt=$(echo "$line" | sed -n 's/.*target=\[([^\]]*\)]\.[^\1/p' | sed 's/ *$//')
    atk=$(echo "$line" | sed -n 's/.*attack=\[([^\]]*\)]\.[^\1/p' | sed 's/ *$//')
    risk=$(echo "$line" | sed -n 's/.*risk=\[([^\]]*\)]\.[^\1/p' | sed 's/ *$//')
    iid=$(echo "$line" | sed -n 's/.*id=\[([^\]]*\)]\.[^\1/p' | sed 's/ *$//')
    ui_box_line "$SUI_WIDTH" " ${C_GRAY}${printf '%-21s' "$ts"}${NC}${C_WHITE}${printf '%-17s' "$tgt"}${NC}${C_PRIMARY}${printf '%-21s' "$atk"}${NC}${C_WARN}${pr
  done
  ui_box_separator "$SUI_WIDTH"
  ui_box_line "$SUI_WIDTH" " ${C_SUCCESS}Full export:${NC} ${C_WHITE}${IOC_FILE}${NC}"
  ui_box_bottom "$SUI_WIDTH"
}
```

Code Snippet 11: Function used to save logs and export IOC files.

7. Testing and Results

The system was tested in a controlled lab environment using Kali Linux and an Ubuntu host. Each attack module was verified to ensure proper alert generation, risk scoring, logging, and dashboard updates.

- Port scanning was validated through TCP SYN packets
- DoS simulation was confirmed using repeated ICMP traffic
- Brute force and malware simulations were verified through generated alerts

The results confirm that the system functions correctly while remaining safe for educational use.

7.1 Attack Simulation Testing

I tested Port Scanning, DoS Attack, Brute Force Login Detection, and Malware Detection using Kali Linux VM and Ubuntu host in a controlled environment.

Port scanning validation was confirmed using Wireshark by observing repeated TCP SYN packets. DoS simulation was validated using repeated ICMP traffic between Kali and Ubuntu. Brute Force and Malware Detection were verified through generated SOC alerts and dashboard responses.

SESSION SUMMARY		INCIDENT REVIEW
▶ LAST INCIDENT		
Attack	DoS Attack	Target 192.168.1.18
Severity	Critical	Risk 100/100
Session Incidents	13	Status Logged
▶ ANALYST TAKEAWAY		
1. Incident was simulated and documented with full SOC evidence.		
2. MITRE ATT&CK mapping and risk scoring applied automatically.		
3. Logs and IOC export are ready for review or presentation.		
▶ NEXT ACTION		
Present Dashboard, IOC Export, and this Summary as the final SOC evidence package.		

No.	Time	Source	Destination	Protocol	Length	Info
4184	657.466937405	192.168.1.19	192.168.1.18	ICMP	98	Echo (ping) request id=0x0009, seq=1/256, ttl=64 (reply in 4185)
4219	658.478594919	192.168.1.19	192.168.1.18	ICMP	98	Echo (ping) request id=0x0009, seq=2/512, ttl=64 (reply in 4220)

4	Frame 4219: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface	0000	8c f8 c5 44 db 12 08 00	27 63 b0 05 08 00 45 00	...	D...	'c...
	Ethernet II, Src: PCSSystemtec_63:b0:05 (08:00:27:63:b0:05), Dst: Intel_L44:da	0010	00 54 94 c5 40 00 40 01	22 6e c0 a8 01 13 c0 a8	...	T..@..	"n...
	Internet Protocol Version 4, Src: 192.168.1.19, Dst: 192.168.1.18	0020	01 12 08 00 f3 2d 00 00	00 02 8e 41 f2 69 00 00	...		..A..
	Internet Control Message Protocol	0030	00 00 ab 4b 04 00 00 00	00 00 10 11 12 13 14 15	...	H.....	
		0040	16 17 18 19 1a 1b 1c 1d	1e 1f 20 21 22 23 24 25	...		..!"
		0050	26 27 28 29 2a 2b 2c 2d	2e 2f 30 31 32 33 34 35	...	&'()*+,-	./012
		0060	36 37		...	67	

Figure 19. Wireshark validation showing repeated ICMP traffic during DoS Attack simulation between Kali Linux and the Ubuntu target system.

7.2 Dashboard and Log Verification

The dashboard was reviewed to confirm accurate display of:

- Incident IDs
- Severity levels
- Risk scores
- MITRE ATT&CK mapping

Additionally, the `/var/log/cybershield/` directory was verified to ensure all logs, reports, and IOC exports were correctly generated after each simulation.

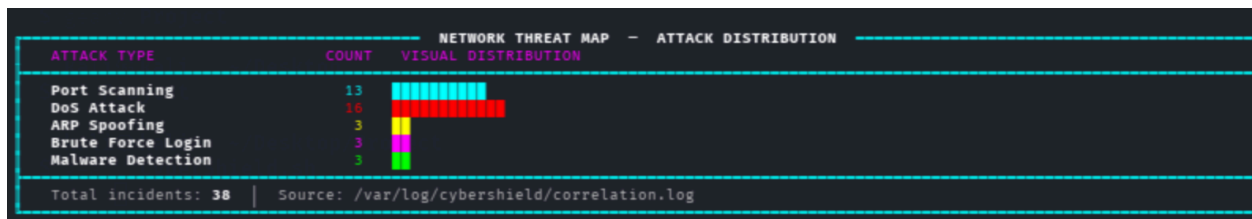


Figure 19. Network threat map for attack simulator.

8. Security Considerations and Future Improvements

CyberShield Simulator is designed for safe educational use and does not execute real attacks or harmful operations. Future improvements may include:

- Integration with SIEM tools such as Splunk
- Real-time alert monitoring
- Threat intelligence integration
- Web-based dashboard interface

9. Conclusion

CyberShield Simulator successfully demonstrates how a Security Operations Center operates in detecting, analyzing, and responding to cyber threats. The project

integrates key SOC components, including network scanning, attack simulation, alert generation, severity classification, dashboard monitoring, and incident logging into a single platform.

Through this project, practical understanding of SOC workflows, incident response, and cybersecurity monitoring has been achieved. The system provides a strong foundation for future development and can be extended to integrate with real-world tools such as SIEM platforms.

10. References

- MITRE ATT&CK. ATT&CK Framework for Enterprise Security Detection and Response. Available at: <https://attack.mitre.org/>
- Nmap. Official Documentation for Network Discovery and Security Auditing. Available at: <https://nmap.org/>
- Wireshark. Official Documentation for Packet Capture and Traffic Analysis. Available at: <https://www.wireshark.org/>
- Kali Linux. Official Documentation and Penetration Testing Platform. Available at: <https://www.kali.org/>
- GNU Bash Documentation. Bash Reference Manual for Shell Scripting and Automation. Available at: <https://www.gnu.org/software/bash/manual/>
- Incident Response Lifecycle. Security Operations Center (SOC) Analyst Workflow and Triage Process.
- Linux Log Management Documentation. Using **/var/log** for Security Monitoring and Incident Investigation.
- Endpoint Detection and Response (EDR) Concepts for Malware Detection and IOC Analysis.